

Kapselung (Encapsulation)

Kriterium	Beschreibung	Wann zu verwenden?
Definition von Kapselung	Kapselung bedeutet in der objektorientierten Programmierung, dass die inneren Details einer Klasse (Daten und Implementierungen) verborgen werden, und nur öffentliche Methoden den Zugriff darauf ermöglichen.	Wenn der Zugriff auf die inneren Daten einer Klasse kontrolliert und nur über explizite Methoden erfolgen soll.
Datenverbergen	Kapselung schützt die Daten einer Klasse, indem sie in privaten Feldern (<code>private</code>) gespeichert werden, um direkten Zugriff von außen zu verhindern.	Wenn Daten vor unbefugtem Zugriff oder unerwünschten Änderungen geschützt werden sollen.
Zugriffskontrolle über Getter und Setter	Anstatt direkt auf Felder einer Klasse zuzugreifen, werden spezielle Methoden (Getter/Setter) bereitgestellt, um auf private Felder zuzugreifen oder sie zu verändern.	Wenn der Zugriff auf Felder kontrolliert werden muss, z. B. durch Validierung oder das Setzen von Bedingungen.
Erhöhung der Wartbarkeit	Durch Kapselung wird die Wartbarkeit des Codes erhöht, da die inneren Implementierungsdetails von anderen Klassen nicht direkt verändert werden können. Änderungen können lokal innerhalb der Klasse vorgenommen werden.	Wenn eine Klasse von außen nicht direkt beeinflusst werden soll und Änderungen an der Implementierung nicht den gesamten Code brechen sollen.
Sichtbarkeitsmodifizierer	Kapselung wird durch die Verwendung von Zugriffsmodifizierern wie <code>private</code> , <code>protected</code> und <code>public</code> erreicht, um zu definieren, welche Teile der Klasse von außen zugänglich sind.	Wenn der Zugriff auf Methoden oder Felder durch Sichtbarkeitsmodifizierer geschützt und gezielt gestaltet werden soll.
Vermeidung von Abhängigkeiten	Kapselung hilft dabei, die Abhängigkeiten zwischen den Klassen zu reduzieren. Durch die Bereitstellung von kontrolliertem Zugriff auf Daten werden unerwünschte Verbindungen verhindert.	Wenn eine enge Kopplung zwischen den Klassen vermieden werden soll und nur notwendiger Zugriff auf die Daten gewährt werden soll.

Kriterium	Beschreibung	Wann zu verwenden?
Verantwortlichkeit von Klassen	Kapselung sorgt dafür, dass eine Klasse nur für ihr eigenes Verhalten und ihre eigenen Daten verantwortlich ist und keine Informationen von außen preisgibt.	Wenn eine Klasse ihre Daten und ihr Verhalten klar von anderen Klassen trennen soll, um Verantwortlichkeiten zu definieren.
Sichtbarkeit von Feldern	Felder einer Klasse sollten möglichst privat sein, um sicherzustellen, dass sie nicht versehentlich oder unerwünscht von außen verändert werden können.	Wenn Daten nicht direkt von außen zugänglich sein sollen, sondern nur durch gezielte Methoden manipulierbar sind.
Verwendung von <code>final</code> bei Feldern	Felder, die nicht verändert werden sollen, können als <code>final</code> deklariert werden, um sie nach der Initialisierung konstant zu machen.	Wenn Felder der Klasse einmal gesetzt und nicht mehr verändert werden sollen.
Schutz vor fehlerhaften Eingaben	Getter- und Setter-Methoden bieten eine Möglichkeit, Eingabewerte zu validieren, bevor sie die privaten Felder ändern.	Wenn sichergestellt werden soll, dass Felder nur gültige Werte erhalten, um Fehler oder Inkonsistenzen zu vermeiden.
Erleichterung der Testbarkeit	Durch Kapselung kann das Testen von Klassen erleichtert werden, da nur die öffentlichen Methoden getestet werden müssen und die Implementierung intern geändert werden kann, ohne dass bestehende Tests angepasst werden müssen.	Wenn die Tests einer Klasse von den internen Details entkoppelt werden sollen.
Verhinderung von direktem Zugriff auf komplexe Datenstrukturen	Kapselung verhindert den direkten Zugriff auf komplexe Datenstrukturen, indem die Daten intern als private Felder verwaltet und nur über kontrollierte Methoden zugänglich gemacht werden.	Wenn der direkte Zugriff auf komplexe interne Datenstrukturen verhindert werden soll, um unvorhersehbare oder fehlerhafte Manipulationen zu vermeiden.
Verwendung von Abstraktion	Durch Kapselung wird eine Abstraktionsschicht geschaffen, die den Benutzer vor den internen Details der Klasse verbirgt und nur die relevanten Informationen bereitstellt.	Wenn eine Klasse ein höheres Maß an Abstraktion erfordert, bei dem der Benutzer nur mit den wichtigsten Methoden interagiert.

Kriterium	Beschreibung	Wann zu verwenden?
Verhindern von zufälligen Modifikationen	Kapselung schützt vor zufälligen oder unerwünschten Änderungen durch die Kontrolle, wie Daten in einer Klasse geändert werden können.	Wenn Änderungen an Daten nur durch explizit definierte Wege und Bedingungen erfolgen sollen.

Zusammenfassung

Kapselung ist eines der grundlegenden Prinzipien der objektorientierten Programmierung und dient dazu, die Implementierungsdetails einer Klasse zu verbergen und nur bestimmte Daten und Methoden öffentlich zugänglich zu machen. Sie schützt die Daten der Klasse vor unbefugtem Zugriff und Manipulation und ermöglicht so eine bessere Kontrolle, Wartbarkeit und Flexibilität.

Wann verwenden?

- Wenn die Daten einer Klasse nicht direkt zugänglich gemacht werden sollen, um sie vor unerwünschten Änderungen zu schützen.
- Wenn der Zugriff auf Felder und Methoden kontrolliert und validiert werden soll.
- Wenn die Implementierungsdetails der Klasse von außen verborgen werden sollen, um den Code flexibler und wartungsfreundlicher zu gestalten.

Vorteile

- **Datenkapselung**
Schützt die Daten der Klasse vor unbefugtem Zugriff und Manipulation.
- **Kontrollierter Zugriff**
Ermöglicht es, den Zugriff auf private Felder und Methoden nur durch explizit definierte Methoden zu steuern.
- **Wartbarkeit und Flexibilität**
Durch Kapselung können Änderungen an der internen Implementierung vorgenommen werden, ohne dass der äußere Code davon betroffen ist.
- **Fehlervermeidung**
Verhindert unvorhersehbare Fehler durch direkte Manipulation von Feldern, insbesondere durch Eingabevalidierungen.

Nachteile

- **Overhead**
Die Implementierung von Getter- und Setter-Methoden für jedes Feld kann zusätzlichen Code und Aufwand verursachen.
- **Zugriffsbarrieren**
Bei einer übermäßigen Kapselung kann es schwieriger werden, auf Daten zuzugreifen, was die Lesbarkeit und Verständlichkeit des Codes beeinträchtigen kann.